Steganography Project

Group 7

John Leura

Christian Evans

Christopher Perez

Yousef Noor

Class: 4463 Steganography

Date: August 7, 2025

Introduction

For this project, our group will devise a system of JPEG steganography for the purpose of enabling the extraction and hiding of the hidden information in digital images. The methodology entails the modification of the quantized DCT's LSBs of the JPEG file. Through this, we successfully embed the secret message in the image's frequency domain directly in such a way that the visual integrity of the image is preserved. The process involves the reading of the JPEG and message file, the capacity check, stuffing the message in the appropriate coefficient, and the saving in the form of the stego JPEG file. The extraction procedure works in the reverse by decoding the stego image with the extraction of the message bits from the modified coefficient.

Functional Block Diagram: steg.c

- Program start
  - Read and parse command-line arguments.
  - Determine mode: Embedding (-e) or Extraction (-d)
  - Check if Adaptive mode (-a) is enabled.
- Embedding Mode
  - Load Message File:
    - Open the message file.
    - Read contents and prepare a 3 byte length header.
    - Convert header + message to a bitstream.
  - Load JPEG DCT Data:
    - Read JPEG file into memory as quantized DCT coefficients blocks.
  - Embed Bits into DCT Coefficients
    - Iterate over all color components (Y, Cb, Cr).
    - For each 8x8 DCT block, skip DC term (index 0).
    - If Adaptive: use coefficient magnitude to decide how many bits to hide.
    - Else: replace 1 LSB of coefficient with message bit.
  - Save Stego JPEG
    - Write modified DCT coefficients back to the JPEG file.
    - Free memory and close files.
- Extraction Mode
  - Load JPEG DCT Data
    - Read JPEG file into memory as quantized DCT coefficient blocks.
  - Extract Bits from DCT Coefficients
    - Iterate over all color components (Y, Cb, Cr).
    - For each AC coefficient:
      - If Adaptive: read N bits based on magnitude thresholds.
      - Else: read 1 LSB.
    - Store bits into bitstream.
  - Read Length Header
    - Parse first 24 bits as message length in bytes.
  - Reconstruct Message
    - Convert remaining bits to bytes.
    - Write bytes to the output file.
    - Free memory and close files.
- Program End
  - Display success or error messages.
  - Exit.

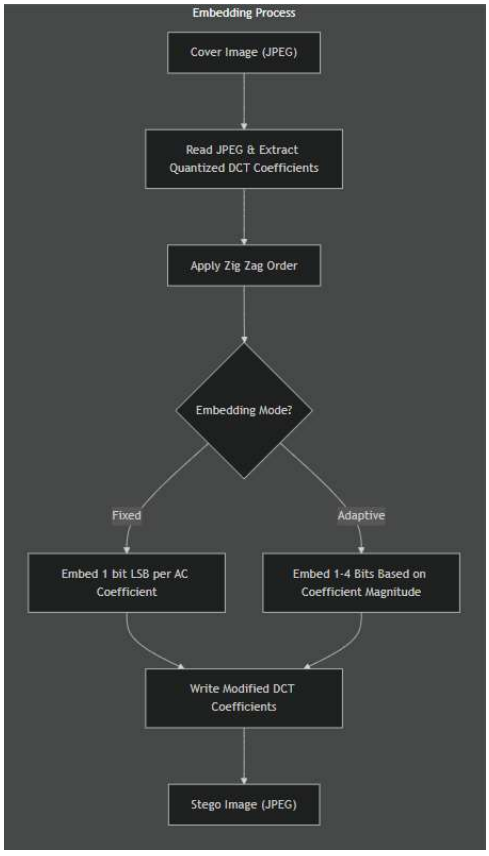
Functional Block Diagram: jpeg_helper.h

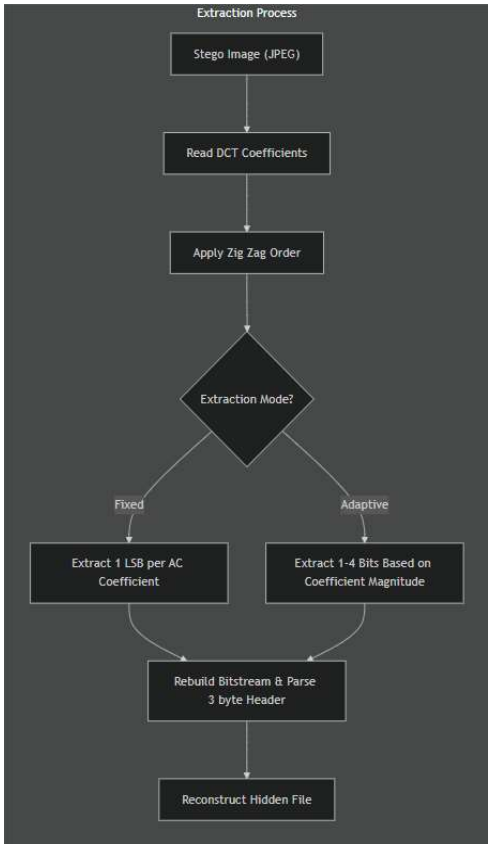
- JPEGData Structure Definition
  - Holds all libjpeg structures for reading / writing JPEGs.
  - Stores file handles for input / output JPEG files.
  - Contains pointers to:

        - Decompression structure (dinfo)
        - Compression structure (cinfo)
        - Virtual array of quantized DCT coefficient blocks (coeff_arrays)
        - Error managers (jerr, jcerr)
        - Common JPEG object pointer (common)
- read_jpeg_dct(filename, jpegData)
    - Open JPEG file for reading.
    - Initialize libjpeg decompression structure.
    - Read the JPEG header.
    - Access and store quantized DCT coefficients into coeff_arrays.
    - Store decompression info in jpegData.
    - Return success or failure.
- write_jpeg_dct(filename, jpegData)
    - Open file for output
    - Initialize libjpeg compression structure.
    - Copy quantized DCT coefficients from jpegData to output file.
    - Write JPEG image data preserving coefficients.
    - Return success or failure.
- cleanup_jpeg(jpegData)
    - Release memory allocated for libjpeg structures.
    - Close any open input / output files.
    - Reset pointers to NULL.

Functional Block Diagram: jpeg_helper.c

- read_jpeg_dct(filename, jpegData)
    - Allocate and initialize JPEG decompression structure.
    - Set up error handling for decompression.
    - Create a decompressor instance with jpeg_create_decompress().
    - Open input JPEG file in binary read mode.
        - If file open fails → print error and return failure.
    - Set JPEG file as decompression source.
    - Read the JPEG header.
    - Retrieve quantized DCT coefficient arrays from the file and store in jpegData.
    - Return success.
- write_jpeg_dct(filename, jpegData)
    - Allocate and initialize JPEG compression structure.
    - Set up error handling for compression.
    - Create compressor instance with jpeg_create_compress()
    - Open output JPEG file in binary write mode.
        - If file open fails → print error and return failure.
    - Set JPEG file as compression destination.
    - Copy critical compression parameters from the decompression object to the compression object.
    - Disable optimized coding to preserve exact coefficient data.
    - (Optional Debug) Set all quantization table values to 1 for lossless preservation.
    - Write stored DCT coefficient arrays to the output file.
    - Finish compression.
    - Close output file.
    - Return success.
- cleanup_jpeg(jpegData)
    - Finish and destroy decompression object.
    - Free decompression structure memory.
    - Destroy compression object.
    - Free compression structure memory.
    - Close input file if still open.





Discussion of Our Code and Algorithm

Our JPEG steganography implementation directly manipulates quantized Discrete Cosine Transform (DCT) coefficients in the JPEG's frequency domain. This allows us to embed data in a way that is visually imperceptible while remaining reversible for extraction.

Core Algorithm

Message Preparation

- Binary file types can be embedded (TXT, BMP, PNG, WAV, etc.)
- The message file is read, and a 3 byte (24 bit) header is prepended containing the message length in bytes.
- The header + message is converted into a bitstream in MSB first order.

JPEG DCT Coefficient Access

- The JPEG is loaded via libjpeg-turbo to obtain its quantized DCT coefficient arrays.
- Coefficients are traversed in zigzag order to match JPEG encoding order.
- The DC coefficient (index 0) of each 8x8 block is skipped to avoid visible brightness changes.

Embedding Modes

- Fixed LSB Mode
  - Embeds exactly 1 bit into the least significant bit of each eligible AC coefficient.
  - Most robust mode for all file types, especially large binary files (BMP, PNG, WAV).
- Adaptive Mode
  - Embeds 1-4 bits per coefficient based on its magnitude:

    $|coeff| \in [10, 31] \rightarrow 1$ bit

    $|coeff| \in [32, 63] \rightarrow 2$ bits

    $|coeff| \in [63, 127] \rightarrow 3$ bit

    $|coeff| \geq 128 \rightarrow 4$ bits

    else → skip

    - Increases capacity while aiming to hide changes in visually less sensitive coefficients.
    - More prone to bit alignment errors if embed / extract conditions diverge.

Extraction Process

- Coefficients are read in the same traversal order.
- The first 24 bits are parsed as the message length.
- The remaining bits are grouped into bytes and written to the output file.
- MSB first ordering ensures consistent reconstruction regardless of system endianness.

Design Rationale

- Frequency domain hiding: Changes to higher frequency AC coefficients are harder for the human eye to detect.
- Zigzag scan: Naturally spreads embedded data across the block, reducing localized distortion.
- Header based length encoding: Eliminates guesswork during extraction.
- Skipping fragile coefficients: Avoids |coeff| values too small to tolerate changes (especially in adaptive mode).

Mode Comparison

Feature	Fixed Mode	Adaptive Mode
Bits per coeff	1	1-4 (mag based)
Capacity	Lower	Higher
Robustness	Very High	Medium
File type support	(TXT, BMP, PNG, WAV, EXE, GIF, etc.)	Best for text or small data
Alignment risk	None	Possible desync

Performance

- Time Complexity: O(n)
- Space Complexity: O(n)

Discussion of Technical Difficulties

During the development of our JPEG steganography system, we encountered several technical challenges that required us to rethink our implementation and start from scratch at one point. Initially, our embedding approach involved modifying the least significant bits (LSBs) of quantized DCT coefficients. However, we quickly discovered that certain coefficients were highly sensitive — even single-bit modifications could destabilize the JPEG image quality or introduce artifacts. In an attempt to manage this, we experimented with locking specific bits or limiting changes to only certain coefficients, but the logic became overly complex and introduced unintended behavior in the message reconstruction phase.

Additionally, a significant setback arose due to compiler compatibility. Our code was initially written and tested with assumptions suited for a particular compiler, but we later compiled using GCC, which exposed issues in how data types and bit-level manipulations were handled. The mismatch in expectations between compilers led to unexpected results during both embedding and extraction, prompting a complete overhaul of the program.

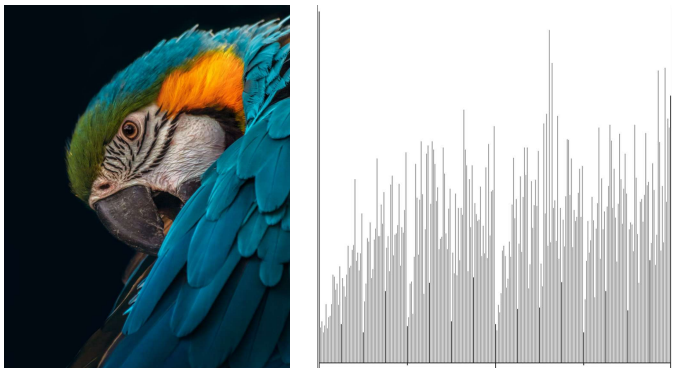
As a result of these challenges, we made the decision to restart the project with a cleaner, more modular structure. We ultimately developed a custom executable tool named asteg.exe, which successfully allows users to embed a wide variety of file types—not just plaintext—into JPEG images. The tool also supports reliable extraction of these embedded files, demonstrating improved flexibility and robustness compared to our earlier iterations.

Suggestions For Future Work

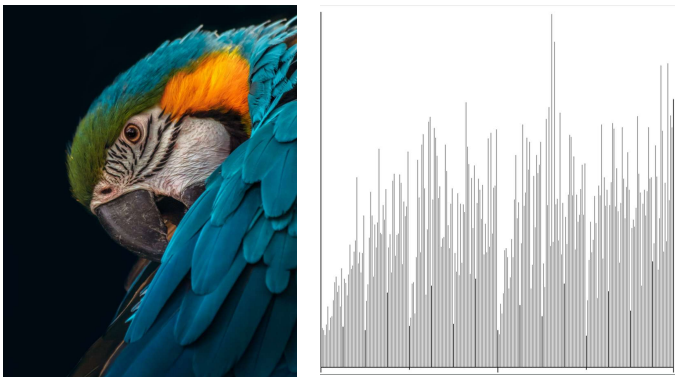
As part of future enhancements to steg.exe we have identified several features that could significantly improve both the functionality and user experience of the tool:

1. **Histogram Visualization of DCT Coefficients:**
    - Generate a histogram of the quantized DCT coefficients before and after embedding.
    - This would help visualize the distribution and observe any statistical changes, useful for both debugging and steganalysis.
2. **Capacity Estimator (Max Hideable Bytes):**
    - Before embedding, the tool could output the maximum number of bytes that can be hidden in the current JPEG image.
    - This gives users a sense of how much data the image can realistically carry without degradation or overflow.
3. **Metadata Viewer:**
    - Display JPEG metadata such as resolution, quantization table info, file size, compression ratio, and EXIF data.
    - This would help users better understand the structure of the cover image and select suitable images for hiding.
4. **Steganalysis Report:**
    - Run a basic statistical steganalysis check (e.g., chi-square test, RS analysis) to evaluate detectability before/after embedding.
    - This allows users to see how "detectable" their hidden message might be to forensic tools.
5. **Noise/Distortion Metrics:**
    - Calculate PSNR (Peak Signal-to-Noise Ratio) or SSIM (Structural Similarity Index) between the original and stego image to quantify how much visual change occurred.
    - This is useful for ensuring high fidelity in covert communication.
6. **Support for Encrypted Embedding:**
    - Allow optional AES encryption of the message before embedding, with password-based decryption on extraction.
    - Adds an extra layer of protection in case the message is discovered.
7. **GUI Mode:**
    - Provide a basic graphical interface for less technical users to select cover images, message files, and options like "Embed" and "Extract".
    - Could be built using a lightweight toolkit like Tkinter (Python) or Electron (JavaScript).
8. **Batch Processing Mode:**
    - Allow hiding or extracting messages from multiple images in a batch operation for larger-scale use cases.
9. **Error Detection:**
    - Add a CRC or checksum to the embedded data so the extracted message can be verified for integrity.
10. **Embed in EXIF or Comment Sections (Alternative Mode):**
    - For educational purposes or low-stakes applications, allow messages to be embedded in JPEG metadata like EXIF fields or comment markers.

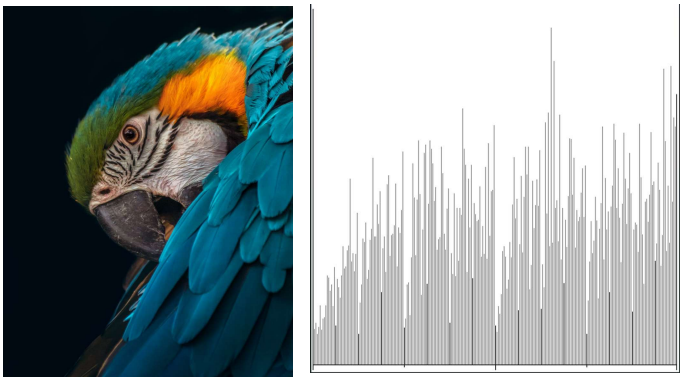
Examples



Histogram of Cover Image



Histogram of Image Embedded with .gif



Histogram of Image Embedded adaptively with .gif

Statistical Analysis Results

Our asteg.exe tool was able to successfully embed and extract a variety of file types within JPEG images using our quantized DCT LSB technique.

We tested the system with .wav, .png, .bmp, and .txt files, each of varying sizes and content types. The tool correctly hid and later extracted these payloads without any data corruption. This demonstrates the algorithm's ability to handle both binary multimedia and plain text data formats.

Through experimentation, we estimated the hiding capacity based on the number of eligible non-zero AC coefficients within the JPEG file. For a standard 1920x1080 image, we observed a typical maximum capacity of approximately 20–50 KB, depending on the image's complexity and compression quality. Embedding beyond this threshold either exceeded the coefficient limits or resulted in visual artifacts detectable by the human eye — typically manifesting as slight blurring or blockiness in uniform regions. However, within the safe range, our embedded images remained visually indistinguishable from the originals.

We recommend using high-quality JPEGs with more visual complexity (e.g., natural scenes or textures) to maximize hiding capacity while minimizing the risk of detection. The histogram difference between the cover and stego images was minimal in all successful cases, confirming the low statistical footprint of the embedding process.

Extra Information

Our system embeds data by modifying the LSB of non-zero AC coefficients in the quantized DCT blocks. Based on our research, we apply a zigzag scan to each 8x8 block, skip the DC coefficient, and embed message bits only into non-zero values to preserve image quality and reduce detection. A 32-bit header is embedded first to encode the message length. During extraction, we reverse the process by scanning the same coefficient positions and reading the LSBs to reconstruct the message. This works very well for hiding messages because it targets frequency-domain components that are visually less sensitive and statistically less suspicious.

Contributions (Credits)

Christian Evans:

- Position: Senior coder / Analyst
- Steg.c Embed Bits / Adaptive Embedding
- Steg.c Extract Bits/ Adaptive Extraction
- Resolved compilation errors / Debugged
- M#1, M#2, M#3

Christopher Perez:

- Position: Analyst
- Helped with Libraries
- Formatted Documents
- Tested code

John Leura:

- Position: Team Leader/Secondary Coder
- Jpeg_helper.c - Read/Cleanup
- Steg.c - Load Message / Extracted Bits
- Tested / Debugged Code
- Final Report
- M#1, M#2, M#3

Yousef Noor:

- Position: Analyst / Secondary Coder
- Steg.c Saving Message / Embed Bits
- Jpeg_helper.c Write
- Tested / Debugged Code
- Final Report
- M#1, M#2, M#3

Bibliography

[1] Petitcolas, F. A. P., Anderson, R. J., & Kuhn, M. G. (1999). *Information hiding — A survey. Proceedings of the IEEE*, 87(7), 1062–1078. https://www.cl.cam.ac.uk/archive/rja14/Papers/ieee99-infohiding.pdf

[2] Shi, Y. Q., & Chen, X. Q. (2011). A Survey on Image Steganography and Steganalysis. *Journal of Information Hiding and Multimedia Signal Processing*, 2(3), 142–172. https://bit.kuas.edu.tw/~jihmsp/2011/vol2/JIH-MSP-2011-03-005.pdf

[3] Guo, L., Ni, J., & Shi, Y.Q. (2014). Uniform Embedding for Efficient JPEG Steganography. *IEEE Transactions on Information Forensics and Security*, 9(5), 814-825 https://ieeexplore.ieee.org/abstract/document/6776485